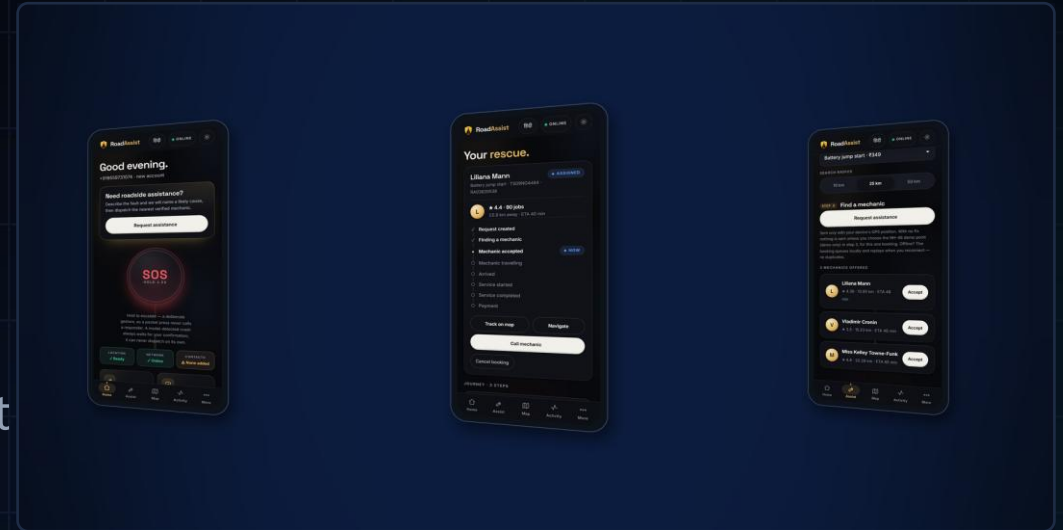


SWE4004 · CLOUD COMPUTING AND APPLICATIONS

# RoadAssist Bharat

AI-Powered · Cloud-Connected · Network-Resilient  
Emergency Mobility Platform

"RoadAssist doesn't stop when the network stops."



## TEAM

V. Saatwik Sairam · P. Sai Nirisha Chowdary

T. V. S. Jignesh · G. Parthavi

## FACULTY

Dr. Nagendra Panini Challa

## INSTITUTION

VIT-AP University

## THE PROBLEM

# A breakdown is not a software problem

...until you realise the app you would use to fix it needs the network you do not have.

21:40

A Swift stops on NH-48. Two hundred kilometres from home, one bar of signal.

21:41

The driver opens a roadside-assistance app. It shows a spinner.

21:44

Still a spinner. There is no way to tell whether anything was sent.

21:52

They give up and start calling numbers from a sticker on the windscreen.

## WHAT ACTUALLY FAILS

- Coverage is worst exactly where breakdowns are most dangerous
- No way to know if a request was sent
- No centralised incident record
- Response time is unmeasured, so it cannot be improved

# One platform, from incident to resolution



Every step is real application state. No manual database editing anywhere in the normal workflow.

# Six things, and one that matters most

## AI-ASSISTED DIAGNOSIS

Rules engine, explainable,  
same table on device and cloud

## DYNAMIC DISPATCH

Ranked, wave-based,  
busy providers excluded

## REAL-TIME TRACKING

Server-sent events,  
measured at 65 ms

## SECURE PAYMENTS

Server-side amount,  
signature-verified webhook

## INCIDENT INTELLIGENCE

Severity, capability,  
tamper-evident audit

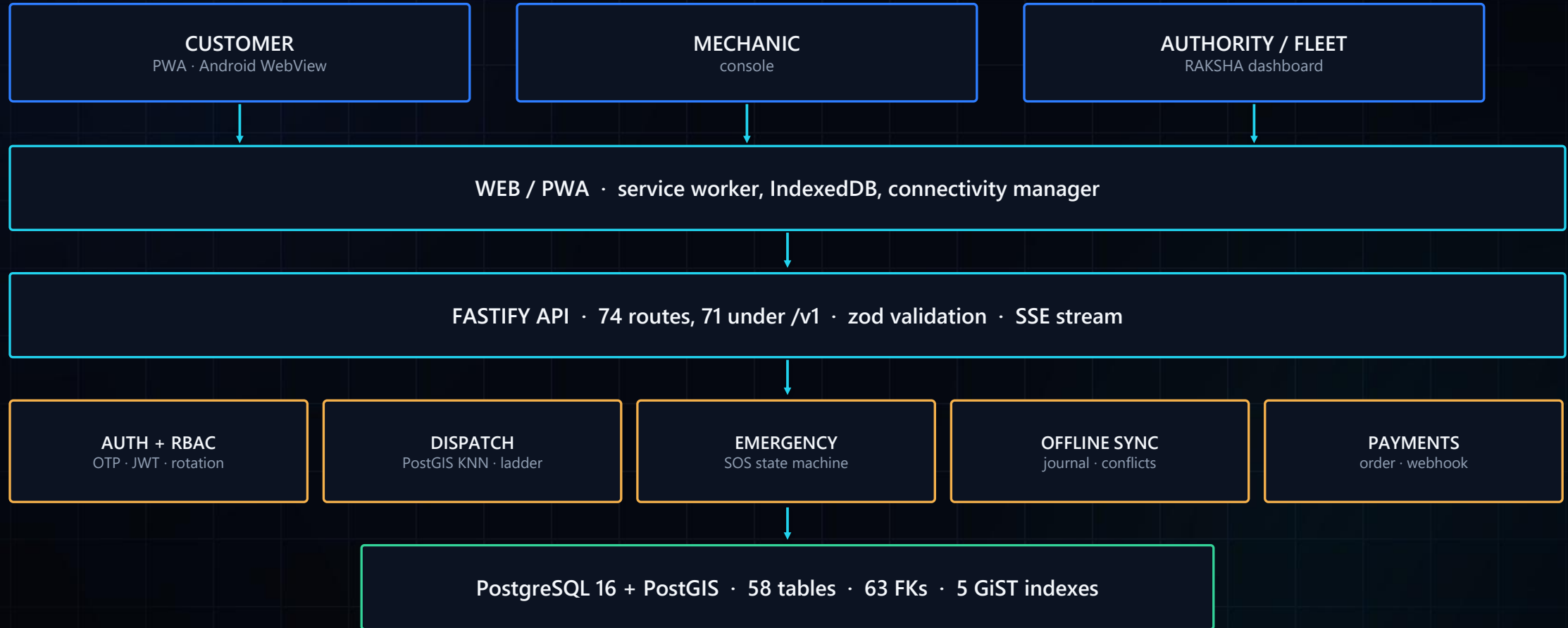
## OFFLINE-FIRST EMERGENCY

SOS with no network,  
stored and forwarded

“RoadAssist doesn't stop when the network stops.”

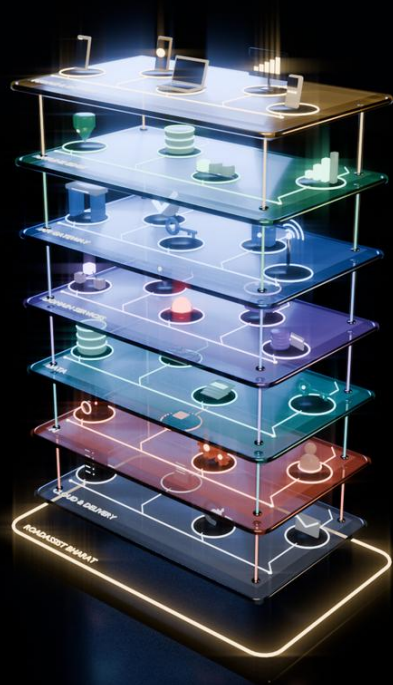
# System architecture

One container, one PostGIS database — live on Render + Neon in Singapore. Everything external is behind an adapter.



NEW · MODELLED IN BLENDER · DRAWN WITH THREE.JS · FED BY THE RUNNING PLATFORM

# Every layer, live in 3D



[app.roadassistbharat.online/layers.html](http://app.roadassistbharat.online/layers.html)

## IMPLEMENTED

- Modelled in Blender 5.2 by a script: 7 layers, 31 parts, each a model of what it stands for; Draco-compressed to 595 KB
- Drawn in the browser with three.js: bloom, reflections, turn it any way, zoom, arrow keys
- Live every 8 s: API round trip, database latency, detections and corridor health
- Three journeys, step by step: a booking, an SOS with no signal, RAKSHA finding a pothole
- Embedded on the project site, the app home and the showcase; opens inside the Android app
- Honest labels: the moving packets are illustration; detection positions are simulated

# Two paths, and the second one is the product



The connectivity manager decides which path runs — **ONLINE**, **LIMITED** or **OFF-GRID** — from measured evidence, never from navigator.onLine alone.

# Off-grid SOS

No internet does not mean the emergency workflow disappears. It means it runs here instead.



## WHAT THE USER IS TOLD

### OFF-GRID SOS CREATED

No network connection detected.

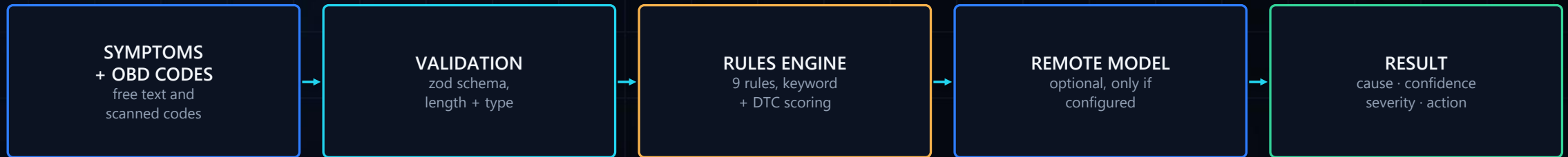
Your emergency information is stored securely on this device and will be synchronized automatically when connectivity returns.

"Nothing has been transmitted."

We never claim emergency services were contacted; the ERSS 112 handoff is a stub. One tap dials 112, 1033 (NHA) or 108. Text my location opens the SMS app; the user presses Send.



# Diagnosis: rules first, model optional



## THE SAFETY ASYMMETRY

A model may make a drivability verdict STRICTER. It can never make one laxer.

The rules result is always computed. A remote answer is used only if it clears AI\_MIN\_CONFIDENCE and does not contradict the rules on safety.

## STATED PLAINLY

This is a deterministic rules engine, not machine learning. The UI labels it "rules-1.0.0", and "LOCAL OFFLINE DIAGNOSIS" on the device.

Trained models DO exist — YOLO11 for road damage: best run YOLO11s, mAP50 0.472; RAKSHA's demo detections come from a YOLO11n, mAP50 0.443. Both measured.

EXPLAINABLE, NOT OPAQUE

# Incident intelligence

Every field below is derived from a rule you can read, not from a weighting nobody can inspect.



## EXPLAINABLE

“Battery discharged or terminals loose” — because these keywords matched, at this confidence

## AUDITABLE

Every prediction writes a model\_predictions row: capability, version, confidence, latency, input HASH (never the text)

## HONEST

Below 55% confidence the UI says so: “At 42% this is a suggestion, not a finding”

## NEVER FABRICATED

No ETA unless a provider is travelling. No location unless GPS returned one.

# Dispatch is dynamic scheduling

Work assigned at run time from a pool, by a score computed from live state, with timeout-driven re-scheduling.



## THE BUG THIS AUDIT FOUND

Dispatch filtered on `is_available` alone — a duty toggle meaning “I am working today”, not “I am free now”. A mechanic already driving to one breakdown kept receiving offers for the next.

Fixed: provider state is DERIVED from live data.

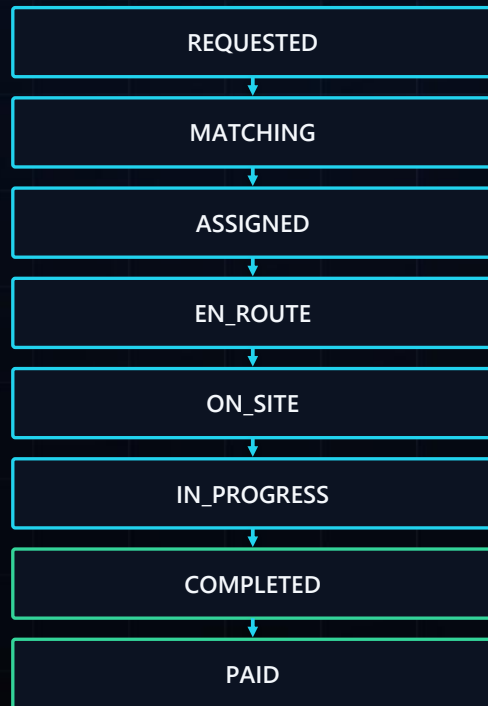
## PROVIDER STATES

|                                 |                                    |
|---------------------------------|------------------------------------|
| <b>OFFLINE</b>                  | off duty — never offered           |
| <b>AVAILABLE</b>                | free — offerable                   |
| <b>OFFERED</b>                  | holding an offer — still offerable |
| <b>BUSY / EN_ROUTE / ON_JOB</b> | committed — never offered          |

# Two guarded state machines

A client sends a command; only the transition table decides whether it is legal. Every change writes a timestamped event row.

**BOOKING — 13 states, 15 commands**    **EMERGENCY — ADR-0005 in code**



## THE RULE THAT IS ENFORCED

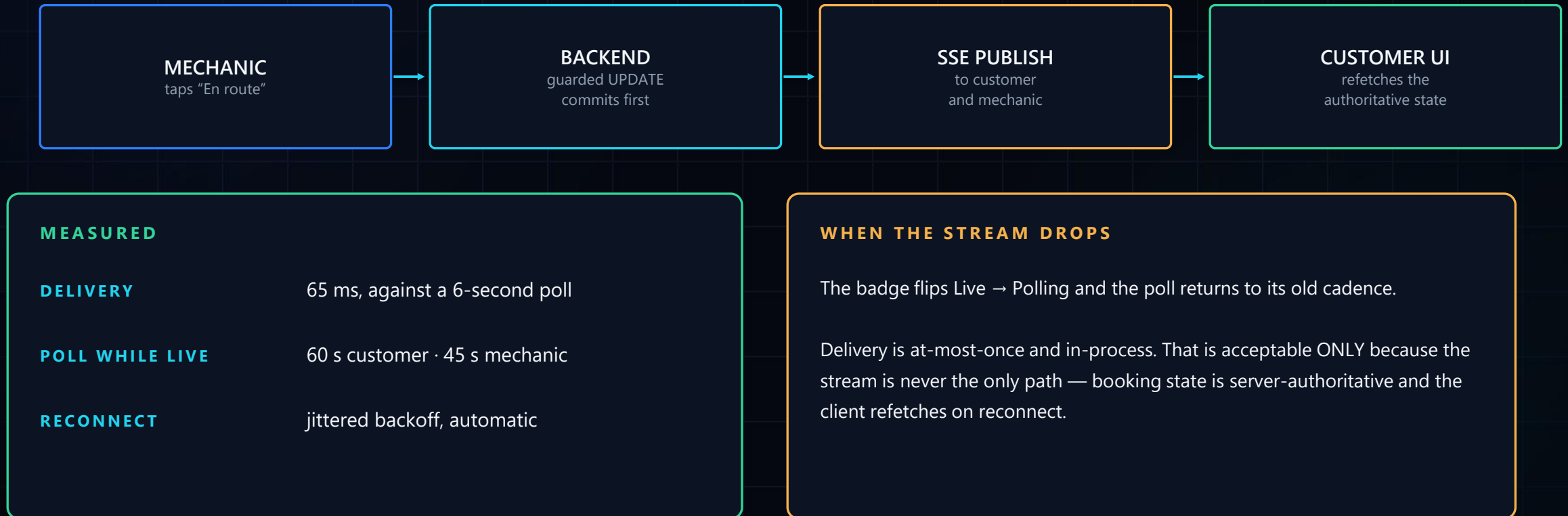
A model raises a SIGNAL. A human confirms an INCIDENT.

“escalate” is reachable only from CONFIRMED. There is no path from a model-detected crash to a dispatched responder that does not pass through a person.

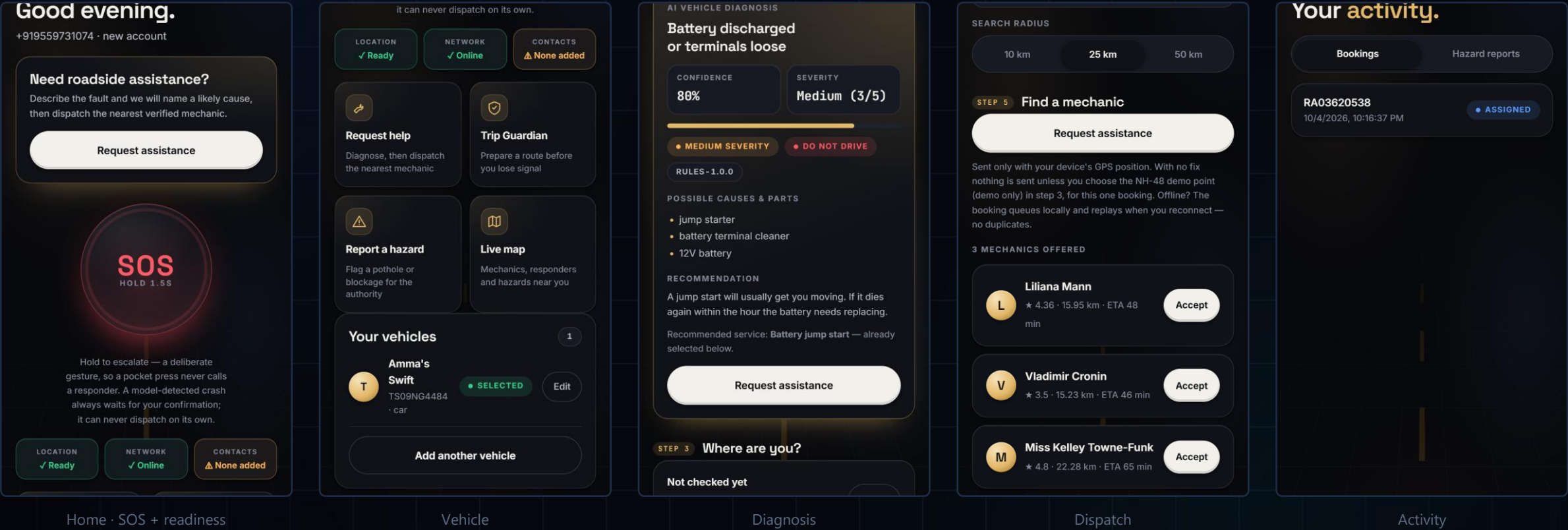
That is one line in a transition table, and it is unit-tested.

# Real-time, but never the source of truth

Server-sent events over one long-lived GET. Persist first, publish second — always.

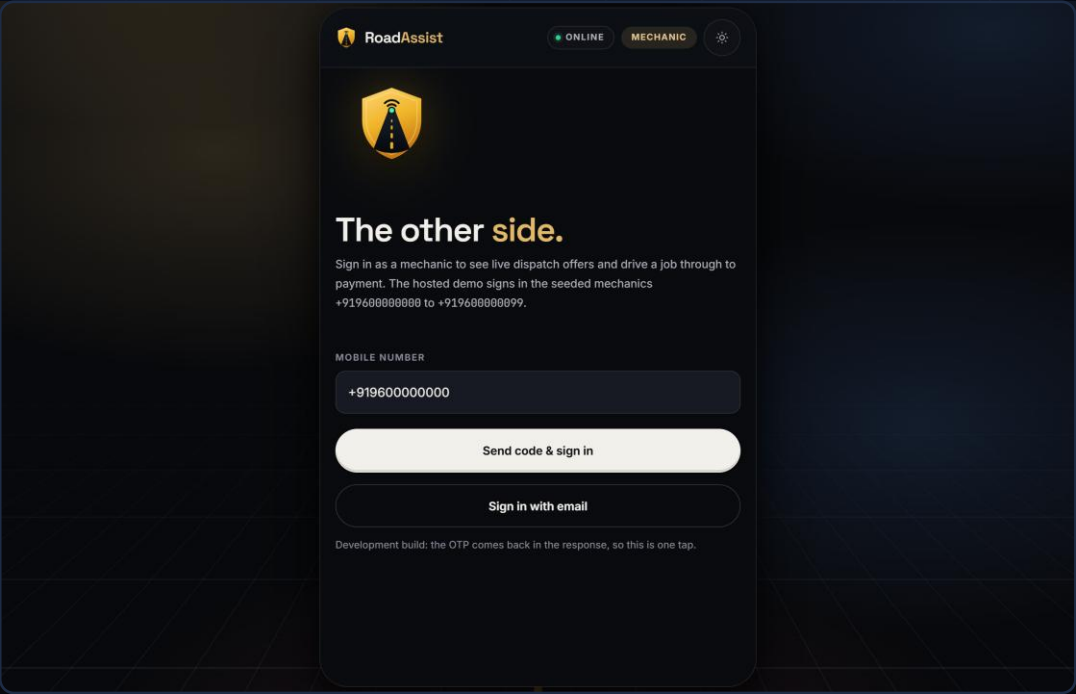


# Customer application



Captured by scripts/capture-screens.mjs against the live API — regenerable in ninety seconds, so they cannot drift from the product.

# Mechanic console and authority dashboard



Mechanic console · phone sign-in, the demo mechanics named on it  
Mechanic: availability · incoming request · accept · decline · en route · arrived · in service · complete — every transition persisted and pushed to the customer.



# Location services from open data

## Near you

OpenStreetMap

Sohna Road, Shivaji Nagar, Gurgaon, Haryana 122001

|                |                                                  |
|----------------|--------------------------------------------------|
| Hospital       | Aarvy Hospital · 290 m<br>Also nearby: 3         |
| Police         | Commissioner Of Police · 540 m<br>Also nearby: 3 |
| Fuel           | IBP Petrol Station · 520 m<br>Also nearby: 3     |
| EV charging    | EV charging (unnamed) · 3.8 km<br>Also nearby: 2 |
| Repair / tyres | KTM · 1.5 km<br>Also nearby: 3                   |

Call

Android · the Near you card on the emulator, real OpenStreetMap data

## WHAT IT ANSWERS

### NEAR YOU · WEB AND ANDROID

Nearest address; the nearest hospital, police, fuel, EV charger and repair shop. Call only where the map has a number, and call 112 under every result.

### TRACKING CARD

Road distance and time beside the straight-line figure, labelled as no live traffic.

### RAKSHA CORRIDOR

Lowest visibility, US AQI at both ends (smog flagged above 150), magnitude 4+ earthquakes in the last 7 days.

## PRIVACY, BY DESIGN

- Server-side only: the page calls this platform, so no visitor's IP reaches a provider
- Coarsened first: about 110 m for an address or route, about 1 km for a search
- Cached (10 min to 24 h) and paced to about one request a second
- Off in development, test and CI: no test run calls a donated service
- Five hosts declared, all outside India: a stated exception to “no PII leaves India”

## SAMPLE LOOKUP AT THE NH-48 DEMO POINT · MEASURED IN TESTING

Sector 12, Gurgaon, Haryana 122001

Aarvy Hospital · 290 m

Road 28.4 km · 33 min

US AQI 195 · Unhealthy

Community map data: a place can be missing or closed, and the card says to call before relying on it. Dispatch still ranks by PostGIS distance; the road time decides nothing.



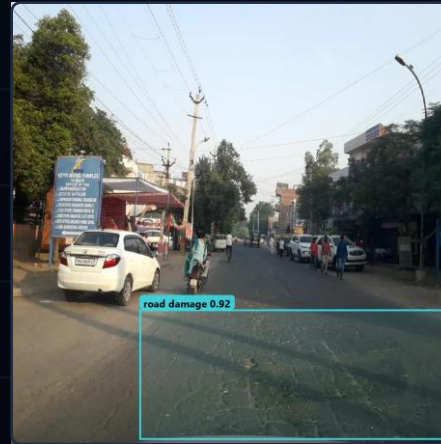
# RAKSHA: the frame the model saw



India\_000914 · 3 road damage, 1 pothole



India\_000053 · road damage 0.89, pothole 0.41



India\_000970 · road damage 0.92

## HOW THE FRAMES WERE MADE

46

RDD2022 India frames

65 / 65

live detections matched

0

unmatched

render\_frames.py re-runs the SAME weights (runs/runs/full) on the images behind the live detections and keeps only boxes whose class and confidence match a listed one. An unmatched detection fails the run.

## ON THE DASHBOARD

### HAZARD PHOTOS

A gallery of the frames and citizen photos, and a photo tile on each detection.

### THE VIEWER

Full screen, with the dataset credit (Arya et al., CC BY-SA 4.0) and "location SIMULATED": RDD2022 images carry no GPS.

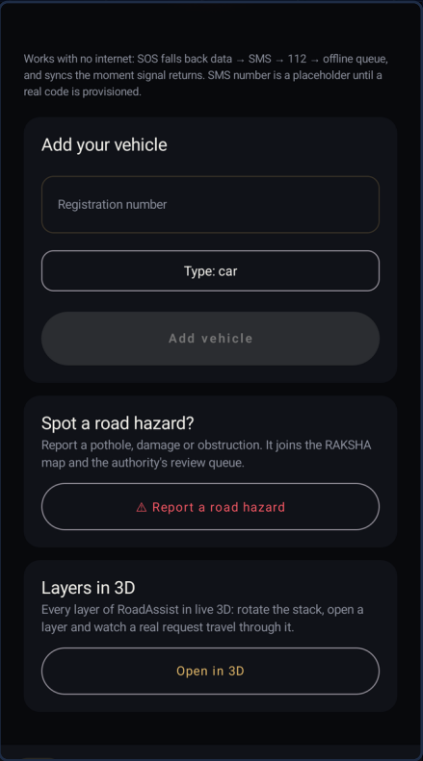
### MAP POPUPS

Each detection's popup shows its frame and opens the same viewer.

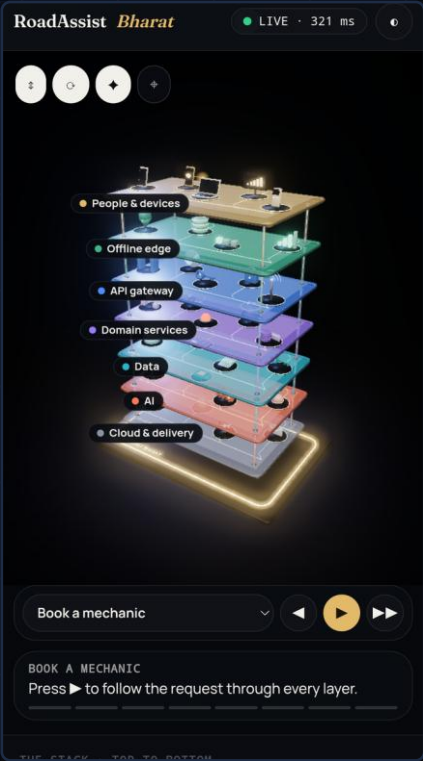
### DISMISS AS FALSE POSITIVE

Admin or officer only, with a written reason, recorded in the audit chain. It is the state machine's cancel: it closes, never dispatches (ADR-0011).

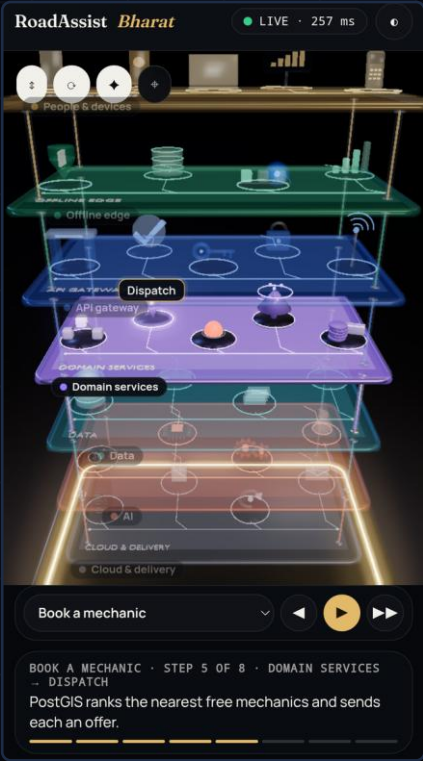
# The Android app, on the emulator



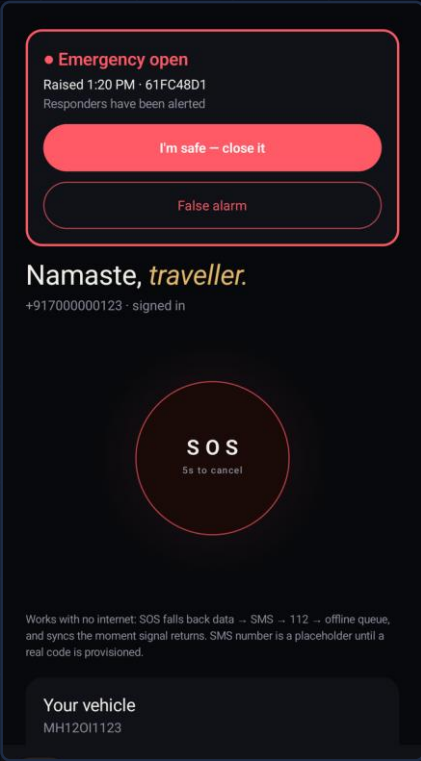
Home · the new Layers in 3D card



Layers in 3D · live, in a WebView



A journey playing · step 5 of 8



An open emergency, found after a restart

179 Android unit tests · 0 lint errors · 8 languages · the emergency card is shown against a local test server

# Three states, three honest answers

## ONLINE

- Everything works
- SOS reaches dispatch
- Live tracking + ETA
- Cloud diagnosis

## LIMITED

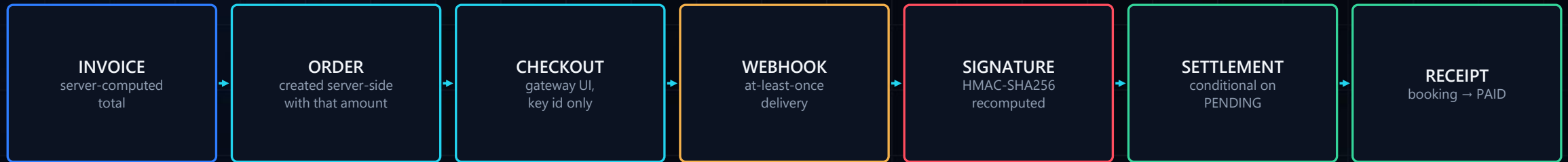
- Requests still attempted
- Anything that fails is STORED
- Slow or flaky path
- User is told it is weak

## OFF-GRID

- SOS stored on device
- GPS still works
- Local diagnosis
- Cached maps · no live anything

The indicator names the state in WORDS as well as colour — status is never carried by tint alone.  
Under every state the primary action stays the same and stays one tap away: SOS.

# The client never decides that money arrived



**AMOUNT**

Never taken from the request — it is the invoice total, and the webhook refuses a mismatch

**TWO PATHS**

Browser callback AND webhook. The webhook is the one that matters: a customer can pay and close the tab

**IDEMPOTENT**

Delivery is at-least-once, so a replayed webhook settles nothing twice

**REFUSES TO BOOT**

Production will not start on a real gateway with no webhook secret

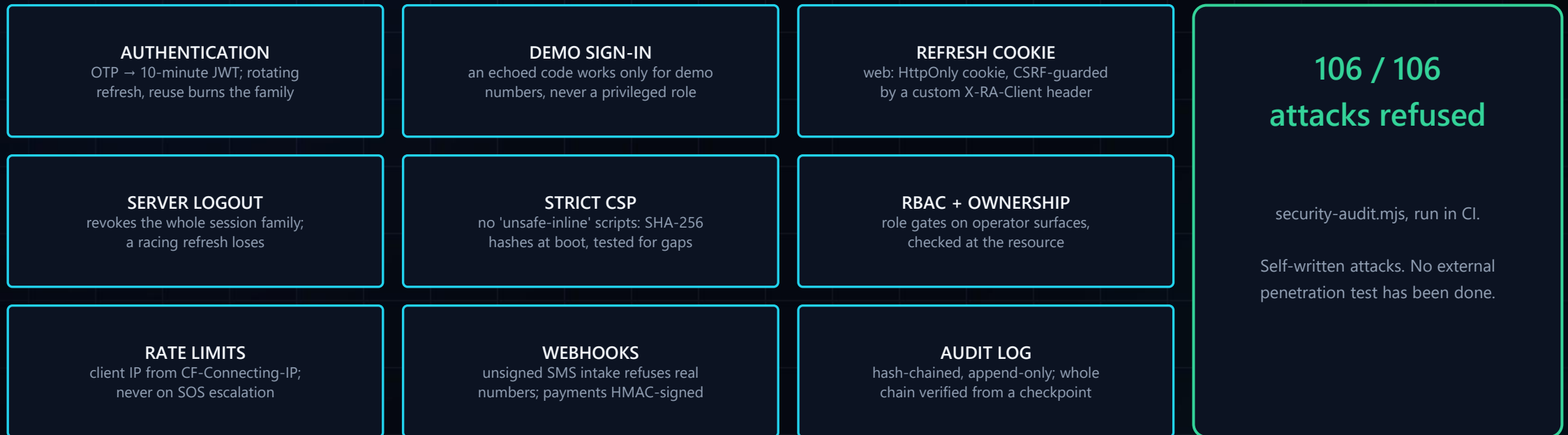
**CHECKED, NOT COUNTED**

22 checks against a local stub of Razorpay's API, no account: forged signature, replay, wrong amount — outside the 1137

NINE LAYERS, AND 106 ATTACKS THAT FAIL

# Security

Refreshed by the September 2026 hardening pass. The suite passes when the platform refuses.



Every source file, the Git history and the live hosts were searched for secrets first: none found; /.env, /.git/config and source maps answer 404.

Medical break-glass needs a role, a LIVE incident and a written reason, and writes a tamper-evident row. Logs redact codes, numbers and coordinates.

Still open, and said so: the 10-minute access token sits in sessionStorage, and style-src keeps 'unsafe-inline'.

EACH NEW TEST FAILS ON THE OLD CODE AND PASSES ON THE NEW

# Found in the recheck, and proved

| WHAT WAS WRONG                                               | BEFORE                                 | AFTER                                            |
|--------------------------------------------------------------|----------------------------------------|--------------------------------------------------|
| Two dispatches of one booking at the same instant            | both sent a wave · 200, 200, 409       | per-booking try-lock: one wins, the rest 409     |
| An SMS SOS recorded a "contacts alerted" step                | texted nobody                          | texts through the confirm guards, count recorded |
| The audit check re-verified only the oldest 2,000–5,000 rows | still said "intact"                    | whole chain from a checkpoint, range stated      |
| A device's image reference, joined to the upload folder      | could read and delete files outside it | refused                                          |
| Sign-out while a refresh was in flight                       | session survived · 8 of 8              | session ended                                    |
| Break-glass response to the responder                        | always "the subject has been notified" | sent · opted out · failed · no number            |

Also fixed: STOP/START opt-out is actually stored and honoured, a cancelled booking withdraws its open offers, the five-contact cap no longer races past five, hazard photos no longer always answer 401 for citizens, and the Android Near you card no longer goes stale after a location-permission grant.

# PostgreSQL 16 + PostGIS

Chosen for one query, and kept for four more reasons.

58

tables

63

foreign keys

142

indexes

5

GiST spatial

7

check constraints

9

migrations

## WHY POSTGIS

“nearest available mechanic to this point” is ONE indexed query with ST\_DWithin doing true metres-on-a-sphere. Without it: fetch every mechanic and compute haversine in app code.

## WHY RELATIONAL

SELECT ... FOR UPDATE is what stops two mechanics taking one job. Transactions are the feature, not a formality.

## CONSTRAINTS

client\_incident\_id UNIQUE is the entire duplicate-emergency defence. Money is integer paise; ratings are CHECK 1–5.

## AUDITABILITY

audit\_log is hash-chained and Postgres RULES block UPDATE and DELETE — an admin cannot quietly edit it.

## VERIFIED

Migrations run from an EMPTY database in this audit, then the full suite. Backup + restore rehearsed: chain intact across 639 entries.

# Where we sit in the cloud

We are a consumer of IaaS and PaaS, and a provider of SaaS to three user classes.

## SaaS

We PROVIDE this: citizen app,  
mechanic console, authority dashboard

IMPLEMENTED

## PaaS

We CONSUME managed Postgres (Neon)  
+ a container platform (Render)

IMPLEMENTED

## IaaS

Compute, network, storage.  
Compose is the local stand-in

PARTIAL

### CHARACTERISTICS WE ACTUALLY EXHIBIT

Broad network access — one API serves a browser, an installed PWA, an Android WebView and a feature phone over SMS.  
Resource pooling — dispatch pools providers and allocates on demand.

### DEPLOYMENT MODEL

Public cloud. Demo: Render + Neon in Singapore — the free tiers offer no India region. Production target: an Indian region, because “No PII leaves India” — logs, backups and crash reports included.



## MODULE 2

# Enabling technologies

Five topics. Four are genuinely built; one is partial and says so.

### Data centre

Compose stack: PostGIS + Redis + Redpanda on a private network; DB port NOT published in production

PARTIAL

### Virtualization

Multi-stage container, non-root (uid 1000), tini as PID 1, healthcheck. Built and run — 321 MB.

IMPLEMENTED

### Web technology

REST over HTTPS with a uniform envelope, server-sent events for real-time, PWA with a service worker

IMPLEMENTED

### Multitenancy

Row-scoped shared schema. 12 cross-tenant attacks fired; all 12 refused.

IMPLEMENTED

### Service technology

Versioned /v1, 74 routes (71 under /v1), zod validation, stable error codes, idempotency keys

IMPLEMENTED

## MODULE 3

# Infrastructure mechanisms

|                        |                                                                                                                |             |
|------------------------|----------------------------------------------------------------------------------------------------------------|-------------|
| Virtual server         | Container image; runs anywhere Docker runs. Full test suite passes AGAINST the image.                          | IMPLEMENTED |
| Cloud storage device   | Postgres volume, upload volume, and client-side IndexedDB with AES-GCM-256 at rest                             | IMPLEMENTED |
| Cloud usage monitor    | Structured JSON logs, per-request correlation id, operation duration/result, /v1/ops/overview with live counts | PARTIAL     |
| Ready-made environment | compose → migrate → seed → start. Reference-only seed for production verified: 0 demo rows.                    | IMPLEMENTED |
| Network perimeter      | DB has no published port; API on loopback behind a proxy; CORS allowlist enforced in production                | PARTIAL     |
| Resource replication   | Stateless API is the precondition and is met. Nothing is replicated.                                           | TARGET      |

## MODULE 4

# The eight fundamental architectures

Two are built. Six are design. I will not show you an animation and call it an autoscaler.

### Workload distribution

The dispatch engine distributes jobs across a provider pool by computed score

IMPLEMENTED

### Resource pooling

Providers are a pool; state is derived and busy members excluded

IMPLEMENTED

### Dynamic scalability

HPA on queue depth. BUILT: the readiness gate — /health returns 503 when the DB is down

TARGET

### Elastic resource capacity

Cluster autoscaler under the pods

TARGET

### Service load balancing

LB with health-checked rotation

TARGET

### Cloud bursting

Modelled evening scenario — labelled as modelled

TARGET

### Elastic disk provisioning

Storage allocated as data arrives

TARGET

### Redundant storage

Replica with failover. Backup procedure documented and rehearsed.

TARGET

Three things block replication today, and each is documented where it is defined: the SSE registry, the rate limiter and the offer sweeper are in-process.

The fix for each is already designed — Redis, and the outbox → Redpanda bus already in the architecture.

# Static vs dynamic scheduling

## STATIC

Decided before run time, on a fixed schedule.

- Offer sweeper — every 10 s (OFFER\_SWEEP\_SECONDS)
- Client booking poll — 6 s, or 60 s while streaming
- SSE heartbeat — every 25 s
- Retention purge — 24 h after a sync

## DYNAMIC

Decided at run time, from live state.

- Dispatch: pool queried at request time
- Score from live distance, rating, workload
- Wave offered; timeout re-schedules to the next wave
- A decline escalates immediately
- Assignment settled under a row lock — exactly one

**RoadAssist's dispatch engine IS dynamic scheduling — it is the clearest Module 4 mechanism the project actually implements.**

## TESTING

# 1137 assertions, all executed

Against a real PostgreSQL + PostGIS and a real Chrome. 22 Razorpay stub checks sit outside this total.

|      |     |                                                                        |
|------|-----|------------------------------------------------------------------------|
| Unit | 451 | state machines, rules, redaction, SMS coordinates, i18n segment budget |
|------|-----|------------------------------------------------------------------------|

|            |     |                                             |
|------------|-----|---------------------------------------------|
| End-to-end | 246 | the whole API journey against real Postgres |
|------------|-----|---------------------------------------------|

|                         |    |                                                   |
|-------------------------|----|---------------------------------------------------|
| Concurrency + real-time | 92 | paces a sequential suite structurally cannot make |
|-------------------------|----|---------------------------------------------------|

|          |     |                        |
|----------|-----|------------------------|
| Security | 106 | attacks that must FAIL |
|----------|-----|------------------------|

|                  |    |                                                     |
|------------------|----|-----------------------------------------------------|
| Gateway security | 58 | webhook signatures, append-only rules, OTP ceilings |
|------------------|----|-----------------------------------------------------|

|                   |     |                               |
|-------------------|-----|-------------------------------|
| Browser / offline | 184 | what only a browser can prove |
|-------------------|-----|-------------------------------|

**1137 passed · 0 failed · 0 server errors across the sweep**

# Every failure has an answer

DETECT → DEGRADE → PRESERVE DATA → RECOVER. Twenty rows in the failure matrix; seven of them here.

| FAILURE             | DETECTED BY                   | USER EXPERIENCE                      | RECOVERY                  |
|---------------------|-------------------------------|--------------------------------------|---------------------------|
| Network lost        | connectivity manager          | off-grid mode; SOS stored locally    | auto on reconnect         |
| Backend unreachable | probe fails, radio up         | LIMITED, then OFF-GRID               | auto                      |
| Database down       | /health → 503                 | safe 500 with a request id           | auto, no restart — tested |
| GPS denied          | geolocation error             | “Unknown — denied”, never a fake fix | next fix                  |
| AI endpoint down    | timeout                       | rules engine — nothing visible       | auto                      |
| Payment fails       | gateway error / bad signature | invoice stays unpaid                 | customer retries          |
| Provider times out  | expires_at passes             | sweeper escalates to next wave       | ladder continues          |

Each row names the suite that proves it. Database loss and backup/restore are now automated CI steps, not prose.

## IMPLEMENTATION STATUS

# What is built, what is partial, what is design

Marked from the code, not from intent.

|                                                                          |                    |                                                                                      |                    |
|--------------------------------------------------------------------------|--------------------|--------------------------------------------------------------------------------------|--------------------|
| <b>AI diagnosis</b><br>rules engine + trained YOLO11 for road damage     | <b>PARTIAL</b>     | <b>Payment</b><br>mock on the demo — never a live account                            | <b>IMPLEMENTED</b> |
| <b>SOS — online</b><br>e2e + concurrency suites                          | <b>IMPLEMENTED</b> | <b>Location services</b><br>server-side, coarsened, cached, off in tests             | <b>IMPLEMENTED</b> |
| <b>SOS — offline</b><br>browser suite drives the whole scenario          | <b>IMPLEMENTED</b> | <b>RAKSHA photo review</b><br>model frames, viewer, false-positive dismiss           | <b>IMPLEMENTED</b> |
| <b>Offline storage</b><br>IndexedDB, AES-GCM-256, survives restart       | <b>IMPLEMENTED</b> | <b>Fleet</b><br>schema + roles exist; no dedicated UI                                | <b>PARTIAL</b>     |
| <b>Store-and-forward sync</b><br>idempotent; duplicate proven impossible | <b>IMPLEMENTED</b> | <b>Analytics</b><br>ops overview with live counts; no BI layer                       | <b>PARTIAL</b>     |
| <b>Dispatch</b><br>ladder, timeout, busy-exclusion, race-safe            | <b>IMPLEMENTED</b> | <b>Cloud autoscaling</b><br>designed; no cluster                                     | <b>TARGET</b>      |
| <b>Real-time</b><br>SSE, measured 65 ms                                  | <b>IMPLEMENTED</b> | <b>Replication / load balancing</b><br>stateless API is the precondition, and is met | <b>TARGET</b>      |

GREEN = code exists, database persists it, UI consumes it, AND a test exercises it. AMBER = some of that. BLUE = architecture only.

# Limitations

In order of importance. Every one of these is also written in the code or the docs.

- 1 The demo is one free-tier Render + Neon instance in Singapore, no India region, no cluster; it sleeps after 15 min idle (keep-awake ping in progress).
- 2 Single instance only: SSE registry, rate limiter and offer sweeper are in-process.
- 3 Location lookups send positions, coarsened to about 110 m, to OpenStreetMap and OSRM servers in Germany/EU. Declared (ADR-0012).
- 4 The ERSS 112 handoff is a stub, and the API response says so. Emergency isolation (ADR-0005) is a design, not a deployment.
- 5 Payment checks run only against a local stub of Razorpay's API, outside the 1137 — never a real account.
- 6 The diagnosis “AI” is a deterministic rules engine. Labelled as such everywhere.
- 7 No load test, no external penetration test, no coverage on the HTTP layer (suites run out-of-process).
- 8 Device encryption protects a storage dump, not script on the same origin — and the UI says exactly that.
- 9 GPS cold start can take minutes and may never fix indoors. The app shows “Unknown” rather than a guess.



FUTURE — LABELLED AS SUCH

# Roadmap

## NEXT

- Redis-backed rate limiting
- Outbox → event bus for SSE fan-out
- WAL archiving (RPO under a day)
- Object storage for photos

## THEN

- Multi-region deployment
- Database read replicas
- External APM + error reporting
- Automated backup schedule
- Self-hosted map services in an Indian region

## LATER

- Predictive maintenance at volume
- Fleet management UI
- Satellite emergency communication
- Mesh networking between devices
- Government emergency-network integration

Satellite, mesh and government-network integration are FUTURE and are labelled that way everywhere in the product — not implemented, not claimed.

# Where the money would come from

Potential, not current. No commercial operation exists — this is a university project.

## B2C

Subscription for individual drivers; pay-per-incident for non-members

## B2B

Fleet operators — uptime is their cost centre; per-vehicle per-month

## B2B2C

Insurers and OEMs bundling roadside cover into an existing policy

## B2G

Highway authorities — RAKSHA road-health data and incident analytics

The unit economics that make it plausible: the platform runs on ZERO paid third-party accounts by default.  
Maps are keyless OpenStreetMap, AI is a local rules engine, and every vendor sits behind an adapter with a working local implementation.

Current status: no revenue, no customers, no commercial deployment. Stated so the model is read as a plan, not a claim.

# What a competitor would have to build

Not “we are better than X” — we have not benchmarked anyone. This is what is genuinely hard here.

## Network resilience

An emergency workflow that runs entirely on the device, with an encrypted journal and idempotent forward-sync. This is the moat.

## Honest degradation

The product never claims something reached the cloud when it did not. That is a design discipline, not a feature you can add later.

## Race-safe dispatch

Row-locked assignment, wave ladder, timeout escalation, derived provider state — proven with ten simultaneous accepts.

## Explainable diagnosis

The same rule table on device and cloud, with a CI test that fails the build if they diverge.

## Auditability

Hash-chained append-only log, verified live in the operations view.

## THE FILM

# RoadAssist Bharat in 30 seconds

Real app footage and real screenshots; the drawn scenes are tagged as illustrations.



## WHAT IT SHOWS

### Request help

the nearest verified mechanic, ranked by PostGIS

### Off-grid SOS

stored on the device; 112 · 1033 · 108 one tap away

### RAKSHA

YOLO11 boxes on real road frames, then authority review

### Under the hood

1137 automated checks, six suites

Also on the showcase: [roadassistbharat.online/media/roadassist-promo.mp4](https://roadassistbharat.online/media/roadassist-promo.mp4)

# Live demonstration

## PART ONE · THE FULL JOURNEY

1. Login
2. Vehicle
3. Breakdown + AI diagnosis
4. Request assistance
5. Dispatch — ranked offers
6. Mechanic accepts
7. Live status, no refresh
8. Service → invoice → payment
9. Review

## PART TWO · THE ONE NOBODY ELSE HAS

1. Network OFF — the UI changes immediately
2. Local diagnosis, labelled
3. Hold SOS → off-grid incident, real GPS
4. "Nothing has been transmitted"
5. CLOSE THE TAB. REOPEN IT.
6. The incident is still there
7. Network ON → automatic sync
8. "SOS synchronized"
9. Replay the reference → duplicate refused

If any step fails, the failure IS the demonstration — nothing here fakes a success.

# RoadAssist Bharat

From breakdown to resolution — even when connectivity fails.

## AI

explainable  
rules engine

## CLOUD

consumed and  
provided

## OFFLINE

the whole emergency  
path on-device

## REAL-TIME

65 ms, and never  
the only path

## SECURITY

106 attacks,  
all refused

## DISPATCH

dynamic, race-safe  
scheduling

“RoadAssist doesn't stop when the network stops.”

1137 assertions · 6 suites · 0 failures · 58 tables · 74 routes · 13 ADRs · every claim on these slides is testable